Ministerul Educației și Cercetării al Republicii Moldova

Universitatea Tehnică a Moldovei

Facultatea Calculatoare, Informatică și Microelectronică

Program: Software Engineering

Operating Systems

Laboratory Work No. 3

The Command Line: Shell, Pipes and Text Filters

Track A: The Real OS, from the Outside (Linux)

Elaborated by:
st. gr. FAF-241
Luchiciov Alexei

Verified by:
Reiţman Patricia

Chișinău – 2026

Contents

1 Introduction 2
2 Data Preparation 2
3 Move Fast, Find Help 3
    3.1 Observations 3
    3.2 Interesting Command Output 3
4 Shell Expansion and Quoting 4
    4.1 Prediction and Results 4
    4.2 Observations 5
5 Redirection, Chaining and Jobs 6
    5.1 Observations 6
6 One-liners 7
    6.1 Q1-Q8 Results 7
    6.2 Full Script Run 8
7 First Bash Script 9
    7.1 Implementation and Test 9
    7.2 Observations 9
8 Conclusion 10

1. Introduction

The purpose of this laboratory work was to become more confident with the Bash command line and to understand what the shell does before a command is executed. During the work I used Tab completion, command history, built-in and external commands, globbing, quoting, redirection, pipes, background jobs, and several text-processing utilities.

The practical work was performed in Ubuntu inside the virtual machine. The main working directory was `~/os-lab3`. The laboratory also used an artificial access log with 500 lines, which was later processed with short one-line pipelines.

2. Data Preparation

The data file for the laboratory was generated with the following loop:

```
mkdir -p ~/os-lab3 && cd ~/os-lab3
for i in $(seq 1 500); do
  echo "2026-10-05 10:$(printf %02d $((i % 60))) user$((i % 7)) \
$([ $((i % 5)) -eq 0 ] && echo FAIL || echo OK) /page$((i % 4))"
done > access.log
head -3 access.log
wc -l access.log
```

Figure 2.1 – Creating and checking the access.log data set

```
2026-10-05 10:01 user1 OK /page1
2026-10-05 10:02 user2 OK /page2
2026-10-05 10:03 user3 OK /page3
500 access.log
```

The check confirmed that `access.log` contained exactly 500 requests, so the data set was ready for the later filtering tasks.

3. Move Fast, Find Help

The first part focused on basic shell navigation, command completion, history, command types, the PATH variable, exit codes, and built-in help.

Tab completion was tested with partial paths. Bash completed `/usr/sh` to `/usr/share/`, and a second Tab showed that many entries were available. The partial path `/et` was completed to `/etc/`.

```
ls /usr/share/
Display all 253 possibilities? (y or n)

cd /etc/
pwd
/etc
```

Figure 3.1 – Tab completion and directory completion

```
type cd
cd is a shell builtin

type ls
ls is aliased to `ls --color=auto'

type echo
echo is a shell builtin

which ls
/usr/bin/ls
```

Figure 3.2 – Output of type and which

```
/usr/local/sbin
/usr/local/bin
/usr/sbin
/usr/bin
/sbin
/bin
/usr/games
/usr/local/games
/snap/bin
/snap/bin
```

Figure 3.3 – Directories contained in PATH

```
nosuchprog
nosuchprog: command not found
echo "exit code: $?"
exit code: 127
```

Figure 3.4 – Exit code of a command that does not exist

3.1. Observations

1. `cd` and `echo` are Bash built-ins. `ls` is configured as an alias that eventually runs the external program `/usr/bin/ls`. `cd` must be a built-in because changing directory has to change the current shell itself; an external child process cannot change the parent shell's working directory.

2. The printed PATH contained 10 entries, although `/snap/bin` appeared twice. The `ls` executable was located in `/usr/bin`.

3. A command that is not found returned exit code `127`.

4. The `ls` option that sorts by modification time is `-t`.

The reverse-history search shortcut `Ctrl+R` searches backward through previously executed commands and allows a matching command to be recalled quickly.

3.2. Interesting Command Output

```
cd is a shell builtin
echo is a shell builtin
ls is aliased to `ls --color=auto'
/usr/bin/ls
```

4. Shell Expansion and Quoting

A small test directory was created with several files. This part demonstrated that many changes to a command line are made by Bash before programs such as `echo` or `ls` are started.

```
touch a.c b.c file1.txt file2.txt notes.txt "alfa beta.txt"
ls
a.c  'alfa beta.txt'  b.c  file1.txt  file2.txt  notes.txt
```

Figure 4.1 – Files created in the play directory

4.1. Prediction and Results

The prediction notes were not preserved in the terminal transcript, so the prediction column below reconstructs the expected reasoning for the exercise. The real-output column is taken from the saved terminal log.

Command	My prediction	Real output	Explanation
echo *.c	The pattern may be printed or expanded to C files.	a.c b.c	Bash expands the `*.c` glob before `echo` runs.
echo file?.txt	Files with one character after `file`.	file1.txt file2.txt	`?` matches exactly one character.
echo [a-b]*	Names beginning with a or b.	a.c alfa beta.txt b.c	`[a-b]` matches the first letter.
echo *.xyz	No matching files.	*.xyz	With no match, Bash leaves the pattern unchanged.
ls alfa beta.txt	It may be treated as one filename.	Errors for `alfa` and `beta.txt`.	The unquoted space creates two arguments.
ls "alfa beta.txt"	The file with a space is listed.	'alfa beta.txt'	Quotes preserve one argument.
echo '$HOME' "$HOME" \$HOME	Literal, expanded, literal.	$HOME /home/alex_007m $HOME	Single quotes suppress expansion; double quotes allow it; backslash escapes `$`.
echo {1..5}; echo $((7*6))	Numbers 1–5 and 42.	1 2 3 4 5 / 42	Brace and arithmetic expansion.
echo "today is $(date +%A), I am $(whoami)"	Day and username.	today is Monday, I am alex_007m	Command substitution inserts command output.
touch log{1..3}.txt && ls log*	Three log files.	log1.txt log2.txt log3.txt	Brace expansion creates three files; `&&` continues after success.

4.2. Observations

1. `echo` itself does not search the filesystem. Bash produced the filenames by expanding the glob before starting `echo`.

2. `ls alfa beta.txt` failed because the space divided the text into two arguments. Two correct forms are `ls "alfa beta.txt"` and `ls alfa\ beta.txt`.

3. Single quotes preserve characters literally. Double quotes still allow expansions such as `$HOME` and command substitution, while protecting spaces.

4. The quotes displayed around `'alfa beta.txt'` only make the embedded space clear; the quotes are not part of the filename.

5. Redirection, Chaining and Jobs

This part explored standard output, standard error, input redirection, conditional command execution, and background jobs.

The first attempt to paste the command that separates stdout and stderr contained terminal bracketed-paste control characters, so that specific attempt did not create the expected `err.txt`. The later redirection commands were executed normally and demonstrated the effect of redirection order.

```
ls /etc /nope > all.txt 2>&1
tail -2 all.txt
xml
zsh_command_not_found

ls /etc /nope 2>&1 > all.txt
ls: cannot access '/nope': No such file or directory
```

Figure 5.1 – Comparing the order of stdout and stderr redirections

```
wc -l < access.log
500

wc -l access.log
500 access.log
```

Figure 5.2 – Input redirection with wc

```
ls /nope 2> /dev/null || echo "failed with code $?"
failed with code 2

sleep 100 &
jobs
[1]+  Running  sleep 100 &
kill %1
[1]+  Terminated  sleep 100
```

Figure 5.3 – Conditional execution and a background job

5.1. Observations

1. In `ls /etc /nope > out.txt 2> err.txt`, normal listing output belongs in `out.txt`, while the error about `/nope` belongs in `err.txt`.

2. With `> all.txt 2>&1`, stdout is redirected first and stderr then follows it, so both streams go to the file. With `2>&1 > all.txt`, stderr keeps pointing to the terminal while stdout goes to the file.

3. `wc -l < access.log` receives data through standard input and no filename argument, so it prints only the number. `wc -l access.log` also prints the filename.

4. `;` always continues; `&&` continues only after success; `||` continues only after failure.

6. Answering Questions with One-liners

The access log was analyzed using short pipelines built from `grep`, `cut`, `sort`, `uniq`, `wc`, `tail`, and related text filters.

6.1. Q1-Q8 Results

Q1. How many requests failed?

```
grep -c FAIL access.log
```

```
100
```

Q2. How many different pages were requested, and which?

```
echo "count: $(cut -d' ' -f5 access.log | sort -u | wc -l)" ; cut -d' ' -f5 access.log | sort -u
```

```
count: 4
/page0
/page1
/page2
/page3
```

Q3. How many requests did each page get?

```
cut -d' ' -f5 access.log | sort | uniq -c | sort -rn
```

```
125 /page3
125 /page2
125 /page1
125 /page0
```

Q4. Which users have the most failed requests, and how many?

```
grep FAIL access.log | cut -d' ' -f3 | sort | uniq -c | sort -rn | awk 'NR==1{m=$1} $1==m'
```

```
15 user5
15 user3
```

Q5. How many requests did user3 make, and how many failed?

```
echo "total: $(grep -c ' user3 ' access.log), failed: $(grep ' user3 ' access.log | grep -c
FAIL)"
```

```
total: 72, failed: 15
```

Q6. Last 3 failed requests, only time and user.

```
grep FAIL access.log | tail -3 | cut -d' ' -f2,3
```

```
10:10 user0
10:15 user5
10:20 user3
```

Q7. Login shells in /etc/passwd.

```
cut -d: -f7 /etc/passwd | sort | uniq -c | sort -rn
```

```
40 /usr/sbin/nologin
6 /bin/false
2 /bin/bash
1 /bin/sync
```

Q8. Compare line counts.

```
ls /etc | wc -l
ls -l /etc | wc -l
```

```
236
237
```

For Q8, the long listing `ls -l` adds a `total` header line, which is why its line count is exactly one larger.

6.2. Full oneliners.sh Run

After correcting the initial save attempt, `bash oneliners.sh` ran successfully. The page counts added up to 500, and the failure counts were consistent with the 100 failed requests.

```
100
count: 4
/page0
/page1
/page2
/page3
125 /page3
125 /page2
125 /page1
125 /page0
15 user5
15 user3
total: 72, failed: 15
10:10 user0
10:15 user5
10:20 user3
40 /usr/sbin/nologin
6 /bin/false
2 /bin/bash
1 /bin/sync
236
237
```

Figure 6.1 – Full run of oneliners.sh

7. My First Bash Script

The final part converted the one-liners into a reusable Bash program named `report.sh`. The script accepts a log filename, checks it, prints total and failed requests, and counts failures for user0 through user6.

7.1. Implementation and Test

```
#!/bin/bash
# usage: ./report.sh access.log

log=$1

if [ ! -f "$log" ]; then
    echo "no such file: $log" >&2
    exit 1
fi

total=$(wc -l < "$log")
fails=$(grep -c FAIL "$log")

echo "$total requests, $fails failed"

for i in $(seq 0 6); do
    n=$(grep -c "user$i FAIL" "$log")
    echo "user$i: $n failures"
done
```

Figure 7.1 – Contents of report.sh

```
./report.sh access.log
bash: ./report.sh: Permission denied
```

```
chmod +x report.sh
./report.sh access.log
500 requests, 100 failed
user0: 14 failures
user1: 14 failures
user2: 14 failures
user3: 15 failures
user4: 14 failures
user5: 15 failures
user6: 14 failures
```

Figure 7.2 – Successful execution of report.sh

```
./report.sh nope.log ; echo "exit code: $?"
no such file: nope.log
exit code: 1
```

Figure 7.3 – Error handling for a missing file

7.2. Observations

1. `#!/bin/bash` is the shebang and selects Bash as the interpreter. Before `chmod +x`, the script did not have execute permission, so direct execution returned `Permission denied`.

2. `./report.sh` is used because the current directory is normally not included in `$PATH`; `./` explicitly selects the file in the current directory.

3. `>&2` sends the error message to standard error, and `exit 1` returns a non-zero status so the caller can detect failure.

8. Conclusion

In this laboratory work I practiced the main command-line mechanisms provided by Bash. I used Tab completion and help commands, inspected built-ins and PATH, and observed how the shell expands glob patterns, variables, quotes, braces, arithmetic expressions, and command substitutions before starting a program.

I also worked with stdout, stderr, redirection order, input redirection, conditional operators, and background jobs. The access.log exercise showed how small text filters can be combined with pipes to answer practical questions using one command line.

Finally, I created `oneliners.sh` and a reusable `report.sh` Bash script. The final script correctly processed 500 requests, found 100 failed requests, printed failure counts for all seven users, and returned exit code 1 when the requested file did not exist. This laboratory helped me understand how Bash connects commands and how simple utilities can be combined into useful programs.